Brute Force Algorithm for Finding the Nearest Weather Monitoring Stations

Aim

To implement the brute-force algorithm that compares all pairs of weather station coordinates to determine the nearest stations.

Problem Statement

A weather monitoring system stores coordinates of multiple weather stations. The objective is to determine the nearest pair of stations by comparing every possible pair using the brute-force method.

Algorithm

1. Store coordinates of all weather stations.
2. Initialize the minimum distance with a large value.
3. Compare every station with every other station.
4. Compute the Euclidean distance.
5. If a smaller distance is found, update the minimum distance.
6. Print the nearest pair.

Explanation

The brute-force algorithm checks every possible pair of weather stations.

If there are **n stations**, then

- Station 1 is compared with all remaining stations.
- Station 2 is compared with stations after it.
- This continues until all possible pairs are examined.

The Euclidean distance formula is

$$Distance = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

The smallest distance obtained is considered the nearest pair.

a) Explain how all pairs are evaluated.

The brute-force algorithm compares every weather station with every other station exactly once.

For example, if there are five stations:

A

B

C

D

E

The comparisons are

A-B

A-C

A-D

A-E

B-C

B-D

B-E

C-D

C-E

D-E

Each comparison calculates the Euclidean distance. After all comparisons are completed, the pair with the minimum distance is selected.

(b) Derive the Time Complexity

If there are **n stations**, the number of comparisons is

$$\frac{n(n-1)}{2}$$

This simplifies to

$$O(n^2)$$

Therefore,

Time Complexity = O(n²)

Space Complexity:

Only a few variables are used.

$$O(1)$$

(c) Inefficiency for Large-Scale Systems

Although the brute-force algorithm is simple and guarantees the correct result, it becomes inefficient when the number of weather stations increases.

For example:

Stations	Comparisons
100	4,950
1,000	499,500
10,000	49,995,000

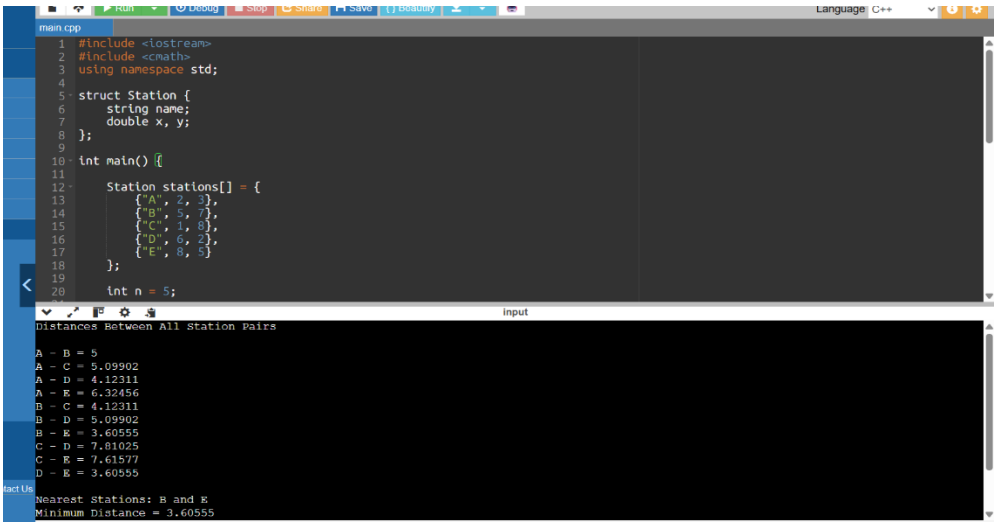
As the number of stations grows, the number of comparisons increases rapidly, resulting in longer execution time. Therefore, brute force is not suitable for large-scale weather monitoring systems.

More efficient algorithms such as the **Divide and Conquer Closest Pair Algorithm** can solve the problem in **O(n log n)** time.

Source Code

```cpp
#include <iostream>
#include <cmath>
using namespace std;

struct Station {
    string name;
    double x, y;
};

int main() {

    Station stations[] = {
        {"A", 2, 3},
        {"B", 5, 7},
        {"C", 1, 8},
        {"D", 6, 2},
        {"E", 8, 5}
    };

    int n = 5;

    double minDistance = 1e9;
    string s1, s2;

    cout << "Distances Between All Station Pairs\n\n";

    for(int i = 0; i < n; i++) {
        for(int j = i + 1; j < n; j++) {

            double distance = sqrt(pow(stations[i].x - stations[j].x,2) +
                                   pow(stations[i].y - stations[j].y,2));

            cout << stations[i].name << " - "
                 << stations[j].name
                 << " = " << distance << endl;
```

Output



Analysis

Advantages

- Easy to understand and implement.
- Always finds the correct nearest pair.
- Suitable for small datasets.

Disadvantages

- Requires comparing every possible pair.

- Execution time grows quadratically.
- Not efficient for thousands of weather stations.

Conclusion

The brute-force algorithm successfully identifies the nearest weather stations by comparing every possible pair of coordinates. It is appropriate for small datasets because of its simplicity and correctness. However, its **$O(n^2)$** time complexity makes it inefficient for large-scale weather monitoring systems, where more advanced algorithms such as Divide and Conquer provide significantly better performance.